

Contents

1	La biología como problema computacional y fundamentos Python	1
1.1	La biología como sistema de procesamiento de información	1
1.2	Las cuatro familias de biomoléculas	2
1.2.1	Ácidos nucleicos: ADN y ARN	2
1.2.2	Proteínas y aminoácidos	2
1.2.3	Carbohidratos y lípidos	3
1.3	Fundamentos de Python para biólogos	3
1.3.1	Tipos de datos primitivos e inmutabilidad	3
1.3.2	Estructuras de datos y colecciones	4
1.3.3	Funciones puras, tipado y control de invariantes	4
1.4	Algoritmos canónicos sobre secuencias biológicas	4
1.4.1	Conteo de nucleótidos y cálculo de contenido GC	4
1.4.2	Complemento reverso de una hebra de ADN	5
1.4.3	Transcripción de ADN a ARN	5
1.4.4	Análisis de ventanas deslizantes (*Sliding Window*)	5
1.4.5	Validación defensiva de entradas	6
1.5	Visualización científica y figuras reproducibles	6
1.6	La operación peligrosa: mutabilidad y desfases de índice	6
1.7	Actividad de depuración: 3 errores en análisis de secuencias	7
1.8	Síntesis operativa y tablas de diagnóstico	7
1.8.1	Tabla de diagnóstico de errores en Python biomédico	7
1.8.2	Tabla de síntesis con preguntas de control	7
1.9	Glosario conceptual	7
1.10	Cierre y conexiones hacia adelante	8
1.11	Fuentes y reproducibilidad	8
	Anexo práctico · Fundamentos de Python y análisis de secuencias	9

Chapter 1

La biología como problema computacional y fundamentos Python

1.1 La biología como sistema de procesamiento de información

Durante siglos, la biología fue una disciplina fundamentalmente descriptiva y observacional. El descubrimiento de la estructura en doble hélice del ADN por Watson, Crick y Franklin en 1953, junto con la formulación del **Dogma Central de la Biología Molecular** por Francis Crick en 1958, transformó radicalmente este paradigma: la vida puede comprenderse y modelarse como un sistema físico de **almacenamiento, transmisión y procesamiento de información digital discreta**.

Esencial

En los seres vivos, las instrucciones necesarias para ensamblar y regular la maquinaria molecular residen en polímeros lineales construidos a partir de alfabetos químicos discretos:

- El **genoma** (ADN) almacena la información hereditaria en un alfabeto cuaternario de cuatro bases nitrogenadas ($\Sigma_{\text{ADN}} = \{A, C, G, T\}$).
- El **transcriptoma** (ARN) actúa como intermediario funcional y catalítico utilizando un alfabeto de ribonucleótidos ($\Sigma_{\text{ARN}} = \{A, C, G, U\}$).
- El **proteoma** ejecuta la inmensa mayoría de las funciones catalíticas, estructurales y de señalización en un alfabeto de 20 aminoácidos estándar (Σ_{Prot}).

El flujo canónico de información biológica discurre desde el ADN hacia el ARN mediante la **transcripción**, y desde el ARN hacia las proteínas mediante la **traducción** según las reglas del código genético universal. No obstante, la biología molecular contemporánea reconoce extensiones críticas a este esquema: la **transcripción inversa** (ADN sintetizado a partir de ARN en retrovirus por la transcriptasa inversa), la **replicación del ARN** (en virus de ARN), las **ribozimas** (moléculas de ARN con actividad catalítica intrínseca) y los **priones** (proteínas capaces de transmitir información conformacional patológica sin mediación de ácidos nucleicos).

La biología como sistema de procesamiento de información: las secuencias y estructuras moleculares constituyen alfabetos digitales discretos que permiten el modelado algorítmico y el análisis computacional riguroso.

La ruta **esencial** cubre la base bioquímica de las cuatro familias de biomoléculas, los fundamentos de Python desde cero (tipos primitivos, inmutabilidad, colecciones, funciones tipadas y excepciones), y los algoritmos canónicos de conteo de nucleótidos, contenido GC, complemento reverso y transcripción; 120–140 minutos. La ruta de **estudio** profundiza en el algoritmo de ventanas deslizantes, el análisis de peligros de mutabilidad, el protocolo de figuras reproducibles (**CON-FIGURES**), la actividad de depuración y el laboratorio práctico; 60–80 minutos adicionales.

Esencial

1.2 Las cuatro familias de biomoléculas

La célula viva organiza su arquitectura química alrededor de cuatro grandes clases de macromoléculas:

Las cuatro familias principales de biomoléculas (ácidos nucleicos, proteínas, carbohidratos y lípidos) presentan propiedades estructurales, metabólicas y computacionales diferenciadas en los sistemas biológicos.

1.2.1 Ácidos nucleicos: ADN y ARN

Los ácidos nucleicos son polímeros de **nucleótidos**. Cada monómero nucleotídico consta de tres componentes unidos covalentemente:

1. Un grupo **fosfato** (PO_4^{3-}), que confiere carga neta negativa y acidez al esqueleto del polímero.
2. Una **aldopentosa** (azúcar de 5 carbonos): β -D-2-desoxirribosa en el ADN y β -D-ribosa en el ARN.
3. Una **base nitrogenada** heterocíclica unida al carbono 1' del azúcar mediante un enlace *N*-glucosídico:
 - **Purinas (anillo bicíclico)**: Adenina (A) y Guanina (G).
 - **Pirimidinas (anillo monocíclico)**: Citosina (C), Timina (T, exclusiva del ADN) y Uracilo (U, exclusivo del ARN).

Los nucleótidos se enlazan consecutivamente mediante **enlaces fosfodiéster** entre el grupo hidroxilo del carbono 3' de un nucleótido y el fosfato en el carbono 5' del siguiente, otorgando a toda cadena de ácido nucleico una **direccionalidad intrínseca** $5' \rightarrow 3'$.

Los monómeros de nucleótidos contienen un grupo fosfato, un azúcar pentosa y una base nitrogenada; el ADN (desoxirribosa, timina, bicatenario) difiere química y funcionalmente del ARN (ribosa, uracilo, monocatenario).

1.2.2 Proteínas y aminoácidos

Las proteínas son los caballos de batalla moleculares de la célula. Se sintetizan como cadenas polipeptídicas no ramificadas de **L- α -aminoácidos** unidos por **enlaces peptídicos** (enlace amida entre el grupo α -carboxilo de un residuo y el grupo α -amino del siguiente).

Cada aminoácido posee un **carbono central α (C_α) tetraédrico** al que se unen cuatro sustituyentes: un grupo amino ($-\text{NH}_2$), un grupo carboxilo ($-\text{COOH}$), un átomo de hidrógeno ($-\text{H}$) y una cadena lateral distintiva ($-\text{R}$). Con excepción de la **Glicina** (cuyo grupo R es un segundo hidrógeno y por tanto es aquiral), los 19 aminoácidos restantes poseen un centro quiral en configuración enantiomérica L.

Los 20 aminoácidos estándar comparten un carbono alfa tetraédrico con grupos amino, carboxilo y una cadena lateral variable R; sus cadenas laterales determinan propiedades fisicoquímicas no polares, aromáticas, polares sin carga, ácidas y básicas.

Clasificación fisicoquímica de las cadenas laterales (grupos R):

- **No polares / Alifáticos (hidrofóbicos)**: Alanina (A), Valina (V), Leucina (L), Isoleucina (I), Metionina (M), Prolina (P, con anillo pirrolidínico rígido) y Glicina (G).
- **Aromáticos**: Fenilalanina (F), Tirosina (Y, anfipática con grupo hidroxilo) y Triptófano (W, anillo indol).

- **Polares sin carga (hidrofílicos neutros):** Serina (S) y Treonina (T, con grupos $-OH$ fosforilables), Cisteína (C, capaz de formar puentes disulfuro covalentes $-S-S-$), Asparagina (N) y Glutamina (Q).
- **Ácidos (cargados negativamente a pH 7.4):** Ácido Aspártico (Asp, D) y Ácido Glutámico (Glu, E), portadores de un segundo grupo carboxilato ($-COO^-$).
- **Básicos (cargados positivamente a pH 7.4):** Lisina (Lys, K), Arginina (Arg, R, con grupo guanidinio ionizado) e Histidina (His, H, cuyo anillo imidazol con $pK_a \approx 6.0$ le permite amortiguar a pH fisiológico).

La clasificación fisicoquímica de aminoácidos asigna el Ácido Aspártico (D) a la clase de cadenas ácidas cargadas negativamente y la Lisina (K) a la clase de cadenas básicas cargadas positivamente. Se reproduce con `verification/chapter_checks.py aa_class D` y `aa_class K`.

1.2.3 Carbohidratos y lípidos

Carbohidratos (Glúcidos): Polihidroxialdehídos o polihidroxiketonas clasificados en monosacáridos (triosas, pentosas, hexosas), disacáridos (sacarosa, lactosa) y polisacáridos. Destacan:

- **Polisacáridos de reserva energética:** Almidón (plantas) y Glucógeno (animales), unidos por enlaces $\alpha(1 \rightarrow 4)$ y ramificaciones $\alpha(1 \rightarrow 6)$, formando hélices abiertas accesibles enzimáticamente.
- **Polisacáridos estructurales:** Celulosa (plantas), unida por enlaces $\beta(1 \rightarrow 4)$ -glucosídicos lineales rígidos reforzados por extensas redes de puentes de hidrógeno intra e intercatenarios.

Lípidos: Macromoléculas anfipáticas o hidrofóbicas no poliméricas. Comprenden ácidos grasos (saturados e insaturados), triglicéridos (almacenamiento energético en tejido adiposo), fosfolípidos (bloques constructores fundamentales de la **bicapa lipídica** de las membranas celulares) y esteroides (colesterol, modulador de la fluidez de membrana y precursor de hormonas esteroideas y sales biliares).

1.3 Fundamentos de Python para biólogos

En Biología Computacional utilizamos **Python 3** como entorno nativo de programación debido a su sintaxis expresiva, ecosistema científico (NumPy, SciPy, Biopython, Matplotlib) y adecuación para modelar estructuras de datos biológicas.

1.3.1 Tipos de datos primitivos e inmutabilidad

Python dispone de tipos primitivos atómicos:

- **Enteros (int):** Representan longitudes de secuencia, conteos de lecturas o índices de posición.
- **Flotantes (float):** Cuantifican concentraciones, porcentajes de identidad, valores E o pesos moleculares.
- **Booleanos (bool):** Indican estados de pertenencia o cumplimiento de aseveraciones biológicas (`True` / `False`).

- **Cadenas de texto (str):** Secuencias de caracteres **inmutables**. Modelan de forma natural polímeros de ADN, ARN y proteínas.

Los tipos primitivos de Python (`int`, `float`, `bool`, `str`) proporcionan representaciones basales donde las cadenas de texto inmutables modelan naturalmente polímeros biológicos.

1.3.2 Estructuras de datos y colecciones

1. **Listas (list):** Secuencias ordenadas y **mutables** delimitadas por corchetes `[]`. Permiten añadir, modificar o eliminar elementos (p. ej., lista de mutaciones identificadas).
2. **Tuplas (tuple):** Secuencias ordenadas e **inmutables** delimitadas por paréntesis `()`. Idóneas para pares de coordenadas genómicas inmutables (p. ej., (`inicio`, `fin`, `hebra`)).
3. **Conjuntos (set):** Colecciones no ordenadas de elementos únicos delimitadas por llaves `{}`. Permiten operaciones de álgebra de conjuntos (unión, intersección, diferencia) en tiempo promedio $O(1)$, idóneas para definir alfabetos biológicos válidos ($\Sigma = \{'A', 'C', 'G', 'T'\}$).
4. **Diccionarios (dict):** Tablas asociativas clave-valor (*hash maps*) en tiempo $O(1)$. Esenciales para codificar tablas de codones, matrices de pesos o mapas de frecuencias nucleotídicas.

Las colecciones de Python corresponden directamente a entidades biológicas: listas para secuencias mutables, tuplas para coordenadas genómicas inmutables, conjuntos para alfabetos y diccionarios para tablas de codones y frecuencias.

1.3.3 Funciones puras, tipado y control de invariantes

En ingeniería de software biomédico, las funciones deben diseñarse como **funciones puras** (sin efectos secundarios) y contar con anotaciones de tipo estático (*type hints*) para prevenir errores en tiempo de ejecución:

Listing 1.1: Funcion tipada con validacion estricta de invariantes.

```
def validate_dna(seq: str) -> bool:
    """Valida que una secuencia contenga exclusivamente bases canonicas A, C, G, T.
    """
    valid_bases: set[str] = set("ACGT")
    upper_seq: str = seq.upper()
    invalid: set[str] = set(upper_seq) - valid_bases
    if invalid:
        raise ValueError(f"Bases invalidas detectadas: {sorted(invalid)}")
    return True
```

Las funciones puras con anotaciones de tipo y validación explícita de invariantes biológicos lanzan `ValueError` al encontrar caracteres nucleotídicos o peptídicos no canónicos.

1.4 Algoritmos canónicos sobre secuencias biológicas

1.4.1 Conteo de nucleótidos y cálculo de contenido GC

El ****contenido GC**** es la proporción porcentual de bases Guanina (G) y Citosina (C) sobre la longitud total de una secuencia de nucleótidos:

$$\text{GC}\% = \frac{\text{Conteo}(\text{G}) + \text{Conteo}(\text{C})}{N} \times 100$$

donde $N = \text{Conteo}(\text{A}) + \text{Conteo}(\text{C}) + \text{Conteo}(\text{G}) + \text{Conteo}(\text{T})$.

Ejemplo resuelto: Para la secuencia $S = \text{'ATGCGATCG'}$ ($N = 9$ nucleótidos):

- Conteos individuales: $A = 2, C = 2, G = 3, T = 2$.
- Conteo $G + C = 3 + 2 = 5$.
- Contenido GC: $GC\% = \frac{5}{9} \times 100 = \mathbf{55.555556\%}$.

Para la secuencia **ATGCGATCG** de longitud 9, el conteo de nucleótidos es A:2, C:2, G:3, T:2 y el contenido GC evalúa exactamente a $5/9 \times 100 = 55.555556\%$. Se reproduce con `verification/chapter_checks.py gc_content ATGCGATCG`.

1.4.2 Complemento reverso de una hebra de ADN

Dado que las dos hebras de la doble hélice de ADN son **antiparalelas** y complementarias (A se aparea con T mediante 2 puentes de hidrógeno, y G con C mediante 3 puentes de hidrógeno), el análisis de la hebra opuesta exige dos operaciones consecutivas:

1. **Complementación:** Sustituir cada base por su pareja de Watson–Crick ($A \leftrightarrow T, C \leftrightarrow G$).
2. **Reversión:** Invertir el orden de la cadena para que la secuencia resultante se lea en la convención canónica $5' \rightarrow 3'$.

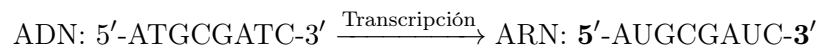
Ejemplo resuelto: Para $S = 5'\text{-ATGCGATC-}3'$:

1. Complemento: $5'\text{-TACGCTAG-}3'$
2. Reversión ($5' \rightarrow 3'$): **$5'\text{-GATCGCAT-}3'$**

El complemento reverso de la secuencia de ADN $5' \rightarrow 3'$ **ATGCGATC** produce la secuencia $5' \rightarrow 3'$ **GATCGCAT**. Se reproduce con `verification/chapter_checks.py reverse_complement ATGCGATC`.

1.4.3 Transcripción de ADN a ARN

Durante la transcripción, la ARN polimerasa sintetiza una molécula de ARN reemplazando la base Timina (T) por Uracilo (U):



La transcripción de ADN a ARN sustituye Timina por Uracilo, convirtiendo **ATGCGATC** en **AUGCGAUC**. Se reproduce con `verification/chapter_checks.py transcribe ATGCGATC`.

1.4.4 Análisis de ventanas deslizantes (*Sliding Window*)

Para estudiar variaciones locales en genomas (como la identificación de islas CpG, orígenes de replicación o dominios de resistencia térmica), se utiliza el algoritmo de ****ventana deslizante****: se evalúa una métrica (contenido GC) en un bloque de longitud fija W que se desplaza a lo largo de la secuencia con un paso S .

Ejemplo resuelto: Para $S = \text{'ATGCGATCG'}$ con ventana $W = 4$ y paso $S = 2$:

1. Ventana 1 (posiciones 0–4): $\text{ATGC} \implies \text{GC} = 2/4 = \mathbf{50.0\%}$
2. Ventana 2 (posiciones 2–6): $\text{GCGA} \implies \text{GC} = 3/4 = \mathbf{75.0\%}$
3. Ventana 3 (posiciones 4–8): $\text{GATC} \implies \text{GC} = 2/4 = \mathbf{50.0\%}$

El análisis de ventana deslizante con ventana 4 y paso 2 sobre **ATGCGATCG** evalúa el contenido GC local en los intervalos 0-4:ATGC:50.0%, 2-6:GCGA:75.0% y 4-8:GATC:50.0%. Se reproduce con `verification/chapter_checks.py sliding_window ATGCGATCG -window 4 -step 2`.

1.4.5 Validación defensiva de entradas

`validate_dna` lanza `ValueError` al detectar bases no canónicas como la 'X' en **ATGCX**, mientras que valida correctamente secuencias biológicas válidas. Se reproduce con `verification/chapter_checks.py validate_dna ATGCX`.

1.5 Visualización científica y figuras reproducibles

La propiedad conceptual **CON-FIGURES** exige que ninguna figura en Biología Computacional se dibuje de forma manual ni mediante capturas opacas:

1. Toda figura proviene de un script Python ejecutable (`make_figures.py`) con dependencias de entorno explícitas.
2. Todo generador estocástico debe fijar una semilla global determinista (`SEED = 42`) para garantizar reproducibilidad bit a bit.
3. Los gráficos científicos deben etiquetar rigurosamente ejes, unidades, leyendas y barras de escala.

Todas las figuras científicas deben generarse de forma reproducible a partir de pipelines programáticos con semillas aleatorias fijas (`SEED=42`) y sin postprocesamiento manual.

1.6 La operación peligrosa: mutabilidad y desfases de índice

Los dos fallos más devastadores y frecuentes en scripts de bioinformática en Python son:

1. **Mutabilidad compartida (*Aliasing*)**: Asignar una lista `b = a` no crea una copia, sino una referencia compartida; modificar `b` altera silenciosamente `a`. Debe usarse `b = a.copy()` o `copy.deepcopy(a)`.
2. **Desfase de indexación 1-based vs 0-based (*Off-by-one*)**: Las bases de datos biológicas (GenBank, Ensembl, GFF) numeran las posiciones de nucleótidos empezando en 1, mientras que Python indexa desde 0 con intervalos semiabiertos `[start, end)`. Un error de indexación desplaza el marco de lectura en un codón o trunca el último aminoácido.

La mutabilidad compartida de listas y el error de desfase de índice entre coordenadas biológicas basadas en 1 y la indexación 0-based de Python representan riesgos críticos en programación biomédica.

1.7 Actividad de depuración: 3 errores en análisis de secuencias

Localice los tres errores conceptuales y sintácticos en el siguiente código:

Listing 1.2: Fragmento con tres errores en procesamiento de secuencias.

```
def compute_gc_and_reverse(dna_seq):
    # Error 1: no convierte a mayusculas ni valida caracteres invalidos
    gc_count = dna_seq.count('G') + dna_seq.count('C')
    gc_percent = (gc_count / len(dna_seq)) # Falta multiplicar por 100

    # Error 2: intenta mutar una cadena inmutable directamente
    dna_seq[0] = 'N'

    # Error 3: complementa pero olvida invertir para el complemento reverso
    comp = dna_seq.translate(str.maketrans('ACGT', 'TGCA'))
    return gc_percent, comp
```

1.8 Síntesis operativa y tablas de diagnóstico

1.8.1 Tabla de diagnóstico de errores en Python biomédico

Síntoma observado	Causa probable	Comprobación y corrección
<code>TypeError: 'str' object does not support item assignment</code>	Intento de mutar una cadena <code>str</code> en lugar de crear una nueva	Usar reemplazo, <code>translate()</code> o convertir a <code>list</code>
<code>KeyError</code> al consultar tabla de codones	Presencia de codón con bases no canónicas o minúsculas	Aplicar <code>.upper()</code> y validar pertenencia con <code>in dict</code>
Contenido GC devuelve 0 o valor truncado	División entera en dialectos antiguos o longitud nula	Validar <code>len(seq) > 0</code> y usar coma flotante
La coordenada genómica extraída está desfasada en 1 base	Confusión entre coordenadas GFF (1-based) y Python (0-based)	Restar 1 al inicio: <code>seq[start-1:end]</code>

1.8.2 Tabla de síntesis con preguntas de control

Entidad biológica	Representación en Python	Pregunta de control
Secuencia de ADN/ARN	<code>str</code> (inmutable, $5' \rightarrow 3'$)	¿Está validado el alfabeto y la dirección?
Coordenadas genómicas	<code>tuple[int, int, str]</code>	¿Se ha convertido de 1-based a 0-based?
Alfabeto canónico	<code>set[str]</code>	¿Es la búsqueda de pertenencia $O(1)$?
Tabla de traducción	<code>dict[str, str]</code>	¿Están cubiertos los 64 codones y los codones STOP?

1.9 Glosario conceptual

- **Dogma central:** flujo de información genética ADN $\rightarrow$ ARN $\rightarrow$ Proteína.
- **Nucleótido:** monómero compuesto de fosfato, pentosa y base nitrogenada.
- **Aminoácido:** monómero proteico con carbono α quiral y cadena lateral R.
- **Enlace fosfodiéster:** unión covalente 3' – 5' en ácidos nucleicos.
- **Enlace peptídico:** unión amida covalente entre aminoácidos.
- **Inmutabilidad:** propiedad de objetos cuyo valor no puede alterarse tras la creación (`str`, `tuple`).
- **Contenido GC:** porcentaje de bases G y C en una secuencia.
- **Complemento reverso:** secuencia complementaria antiparalela leída 5' $\rightarrow$ 3'.
- **Ventana deslizante:** análisis de subintervalos consecutivos de longitud fija W .
- **Off-by-one:** error de desfase en límites de indexación por discrepancia de base.

1.10 Cierre y conexiones hacia adelante

Este capítulo inicial establece la base computacional en Python y la representación digital de secuencias requerida a lo largo de todos los capítulos de Biología Computacional.

En el próximo capítulo (**BC-CH02: Biomoléculas como objetos computables**), eleveremos estas representaciones básicas hacia clases y estructuras orientadas a objetos, modelando propiedades fisicoquímicas, pesos moleculares y perfiles avanzados.

1.11 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas docentes (`20250911_bq_esencial.pptx`, Álvaro Serrano Navarro), en el tratado de Alberts et al. (2014) y en la especificación oficial de Python 3 (Van Rossum & Drake, 2009). Todos los algoritmos y cálculos se reproducen de forma determinista mediante `verification/chapter_checks.py`.

Anexo práctico · Fundamentos de Python y análisis de secuencias

Pregunta, producto y separación de materiales

¿Puede implementar y verificar en Python estándar los algoritmos fundamentales de conteo nucleotídico, contenido GC, complemento reverso, transcripción, análisis de ventanas deslizantes y validación defensiva de secuencias sin depender de paquetes externos opacos? Este laboratorio responde que sí, de forma totalmente reproducible.

El producto individual contiene: el cálculo manual y ejecutado del contenido GC y complemento reverso para la secuencia ancla, la perturbación de parámetros en ventanas deslizantes, el control de excepciones en la guarda de dominio, y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere datos externos ni conexión a la red. Emplea `verification/practice_checks.py` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Versión de Python y comprobación de scripts
2 · Cálculo ancla (GC y RevComp)	20–25	Métricas calculadas a mano vs script
3 · Transcripción y conteos	15–20	Conversión ADN→ARN y mapa de bases
4 · Perturbación de ventanas	15–20	Análisis con ventana $W = 6$ y paso $S = 3$
5 · Guarda de dominio	10–15	Captura y verificación de ValueError
6 · Clasificación de aminoácidos	5–10	Asignación fisicoquímica de residuos
7 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 1.3: Comprobación del intérprete Python.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: contenido GC y complemento reverso

Dada la secuencia de ADN $S = 5'\text{-ATGCATGCGTCG-}3'$ ($N = 12$ bases):

1. Cuente manualmente cada una de las cuatro bases.
2. Calcule el contenido GC exacto: $\text{GC}\% = \frac{G+C}{N} \times 100$.
3. Obtenga el complemento reverso en dirección $5' \rightarrow 3'$.

Listing 1.4: Ejecutar el calculo ancla.

```
python3 verification/practice_checks.py anchor
```

La secuencia ancla ATGCATGCGTCG de longitud 12 produce un contenido GC de 58.333333% y un complemento reverso CGACGCATGCAT. Se reproduce con `verification/practice_checks.py anchor`.

3 · Transcripción y perfiles de nucleótidos

Transcriba la secuencia ancla a ARN y compruebe los conteos:

Listing 1.5: Transcripcion a ARN y conteo.

```
python3 verification/chapter_checks.py transcribe ATGCATGCGTCG
python3 verification/chapter_checks.py nucleotide_counts ATGCATGCGTCG
```

4 · Perturbación de ventanas deslizantes

Evalúe el contenido GC local sobre la secuencia ancla utilizando una ventana de longitud $W = 6$ y un paso de desplazamiento $S = 3$:

Listing 1.6: Perturbacion de longitud y paso de ventana.

```
python3 verification/practice_checks.py perturbation --window 6 --step 3
```

La perturbación de ventana deslizante con ventana 6 y paso 3 sobre ATGCATGCGTCG produce las ventanas 0-6:ATGCAT:33.3%, 3-9:CATGCG:66.7% y 6-12:GCGTCG:83.3%. Se reproduce con `verification/practice_checks.py perturbation -window 6 -step 3`.

5 · Guarda de dominio y control de excepciones

Verifique que su función de validación rechaza correctamente secuencias contaminadas con caracteres no canónicos:

Listing 1.7: Guarda de dominio ante bases no canonicas.

```
python3 verification/practice_checks.py domain_guard --seq ATGCZ
```

La guarda de dominio rechaza la secuencia no válida ATGCZ lanzando un `ValueError` que identifica el carácter inválido 'Z'. Se reproduce con `verification/practice_checks.py domain_guard -seq ATGCZ`.

6 · Clasificación fisicoquímica de residuos

Consulte la clase fisicoquímica del Ácido Aspártico (D) y la Lisina (K):

Listing 1.8: Clasificación de aminoácidos.

```
python3 verification/chapter_checks.py aa_class D
python3 verification/chapter_checks.py aa_class K
```

7 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin requerir librerías externas ni acceso a la red.

Rúbrica de calificación ponderada

La evaluación cuantitativa sobre 10 puntos se distribuye según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual exacto del contenido GC y complemento reverso contrastado frente al oráculo (Bloque 2).
- **2.0 puntos (20%):** Análisis de ventanas deslizantes y control defensivo de excepciones (Bloques 4 y 5).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre representación discreta de biomoléculas e inmutabilidad.

Lista de autocorrección

- | | |
|--|--|
| • Verifiqué que mi versión de Python es ≥ 3.9 . | • Registré las 3 ventanas con $W = 6$ y $S = 3$. |
| • Mi cálculo manual de GC% coincidió con 58.333333%. | • Verifiqué que <code>ATGCZ</code> lanza <code>ValueError</code> . |
| • El complemento reverso calculado fue <code>CGACGCATGCAT</code> . | • Comprobé las clases fisicoquímicas de D y K. |
| • Comprobé la transcripción sustituyendo T por U. | • Redacté la defensa conceptual individual. |

Defensa conceptual

En 100–150 palabras, explique por qué las cadenas de texto (`str`) en Python son inmutables y qué ventajas de seguridad aporta esta propiedad al manipular secuencias genómicas, contrastándolo con los riesgos de mutabilidad compartida de las listas (`list`).

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de bioquímica esencial (20250911_bq_esencial.pptx) y se valida al 100% mediante `verification/chapter_checks.py` y `verification/practice_checks.py`.